Data Structure and Algorithms

REPORT ASSESSMENT 3

Trimester 1, 2024

Ngoc Thanh Tam DANG _ 90028406

1. Introduction

This program is a contact management system designed to organize and manage users' contacts effectively. It provides functions to add, delete, update, search, sort, and filter contacts. The system is implemented in the Python programming language using object-oriented principles and data structures.

The purpose of the program is to provide users with a convenient way to store, manage, and access their contact information. In today's digital age, where individuals interact with multiple contacts across various platforms, having a centralized system to organize contact details has become necessary. The program aims to streamline contact management tasks, reduce manual effort, and improve productivity.

The program's target audience includes individuals, professionals, and organizations that need to manage contact information effectively. This can range from individual users looking to organize their personal and professional contacts to businesses and organizations dealing with large databases of customers, clients, or members. The program serves users across different domains and is designed to be user-friendly, making it accessible to a wide range of users regardless of technical expertise.

2. Dependencies

The program is based on standard Python libraries and does not have any external dependencies. It uses basic data structures and functions provided by the Python language itself. Therefore, there is no need to perform additional installation steps other than installing Python on your system.

Since the program does not require any external dependencies, there is no need for installation instructions. However, users should ensure that they have Python installed on their system to run the program. Most modern operating systems come with Python pre-installed, but if it isn't available, you can easily download and install Python from the official Python website (https://www.python.org/ downloads/).

3. Terminologies

Hash table

A hash table is a data structure that stores key-value pairs where each key is mapped to a unique index in an array using a hash function. It provides the ability to insert, delete, and retrieve elements efficiently

Collision

Collisions occur in hash tables when two or more keys are hashed to the same index. This situation needs to be handled to avoid data loss and maintain the integrity of the hash table.

Stringing

Chaining is a technique used to handle collisions in hash tables. It involves storing multiple key-value pairs at the same index in a hash table. Each index contains a linked list (or other data structure) of elements that hash into the same index.

UML (Unified Modeling Language)

UML is a standardized modeling language used in software engineering to visualize, specify, construct, and document software system artifacts. It includes many different types of diagrams, such as class diagrams, sequence diagrams, and activity diagrams, to represent different aspects of a system's structure and behavior.

Class diagram

A class diagram is a type of UML diagram that represents the structure of a system by showing classes, properties, methods, and relationships between them.

Readme file

A readme file is a text file that contains information about a project, often including installation instructions, usage, and other related details. It serves as a guide for users and developers to understand and interact with the project.

Time complexity

Time complexity is a measure of the amount of time an algorithm takes to execute as a function of the length of the input. It provides an estimate of the computational resources required by the algorithm to solve the problem.

Space complexity

Space complexity is a measure of the amount of memory space an algorithm requires to execute as a function of the length of the input. It provides an estimate of the storage resources required by the algorithm to solve the problem.

Origin of matrix output

The original release matrix is a tool used in software development to map and track the relationships between different products, albeit with limitations such as requirements, design components, and experimental testing. case test. It helps ensure that all requirements are met by corresponding tests and vice versa, facilitating thorough software testing and validation.

Comments

Comments are added to the code by the developer to provide appropriate solutions, descriptions, or clarifications about code logic, algorithms, or detailed declarations. We've improved the ability to read and understand code for developers working on the codebase of the future.

4. Future Direction

User authentication

Implement user authentication to secure the contact management system, allowing users to log in with their credentials and securely access their contacts.

Data encryption

Introducing data encryption techniques to encrypt communications stored in the system, enhancing the privacy and security of users' sensitive data.

User interface improvements

Enhance the user interface with modern design principles and interactive features to improve user experience and usability.

Import/export function

Added functionality to import contacts from external sources (e.g. CSV files, vCard files) and export contacts to various formats for backup and interoperability with other applications.

Contact relationship management

Expands the system to support relationship management between contacts, allowing users to categorize contacts based on their relationship (e.g. family, friends, colleagues) and visualize their connection between contacts.

Integration with Communication Platforms

Integrate the system with communication platforms (e.g. email, messaging apps) to enable seamless communication with contacts directly from the app.

Cloud Sync

Implement cloud sync features to synchronize contacts across devices and platforms, ensuring users can access their contacts from anywhere.

Advanced search and filtering

Enhance your search and filtering capabilities with advanced options (e.g. fuzzy search, advanced filters) to quickly locate specific contacts based on various criteria.

5. Class UML diagram

Readme file:

The readme file should provide information about the purpose of the project, how to use the contact management system and any other relevant details for users and developers.

UML class diagram:

Below is a UML class diagram that shows the classes used when implementing a contact management system:

Class relationships and associations

Class relationships and associations within the system are clearly described in UML class diagrams. Each class represents a specific entity (e.g. Contact, ContactHashTable) with its properties and methods, and the relationships between the classes illustrate how they interact and collaborate within the system. Associations such as composition, composition, and inheritance are used to represent different types of relationships between classes.

6. Time complexity:

Add contact action

O(1) on average for hash table insertion, but can be reduced O(n) in the worst case due to collision.

Contact deletion action

O(1) on average to clear the hash table, but can be reduced O(n) in the worst case due to collision.

Active contact search

O(1) on average for hash table lookup, but can be reduced O(n) in the worst case due to collision.

Contacts sorting operation

O(nlogn) for merge sort.

Contact filtering activities

O(n) to iterate through contacts and filter based on group.

7. Space complexity:

Hash table

O(n) space for storage n contact address, where n is the number of contacts.

Merge Sort

O(n) space for auxiliary arrays during sorting.

Evaluate efficiency and scalability

Contact management systems provide efficient operations for adding, deleting, searching, sorting, and filtering contacts.

Performance may degrade in the worst case of hash table collisions, but overall performance is still satisfactory for typical use cases.

The system is designed to handle a large number of contacts efficiently and to scale to the size of the contact database.

8. Traceability matrix

Feature	Testing Procedure
Add Contact	Unit tests: test_add_contact() Integration tests: test_add_contact_integration()
Delete Contact	Unit tests: test_delete_contact() integration tests: test_delete_contact_integration()
Update Contact	Unit tests: test_update_contact() integration tests: test_update_contact_integration()

Search Contact	Unit tests: test_search_contacts() integration tests: test_search_contacts_integration()
Sort Contact	Unit tests: test_sort_contacts() integration tests: test_sort_contacts_integration()
Filter Contact	Unit tests: test_filter_contacts() integration tests: test_filter_contacts_integration()
Display Contact	Unit tests: test_display_contacts() integration tests: test_display_contacts_integration()

Evaluate test coverage and completeness

Test coverage is comprehensive, with unit tests covering individual functions and integration tests verifying interactions between components.

The testing process ensures that each feature functions as expected and meets specified requirements.

Continuous integration and automated testing help maintain high test coverage and ensure system reliability.

9. Comments on Code

Chained hash table

The decision to use a chained hash table represents a thoughtful approach to efficiently store and retrieve contacts while handling collisions. Chaining allows multiple contacts to be stored at the same index, ensuring that contacts with different phone numbers but the same hash value can coexist in the data structure without conflicts.

Object-oriented design principles

Adherence to object-oriented design principles, such as modularity, encapsulation, and reusability, demonstrates a structured and organized approach to development. software development. By encapsulating contact-related functionality in classes such as DSAcontact and DSAcontactHashTable, the code promotes code reuse and maintainability.

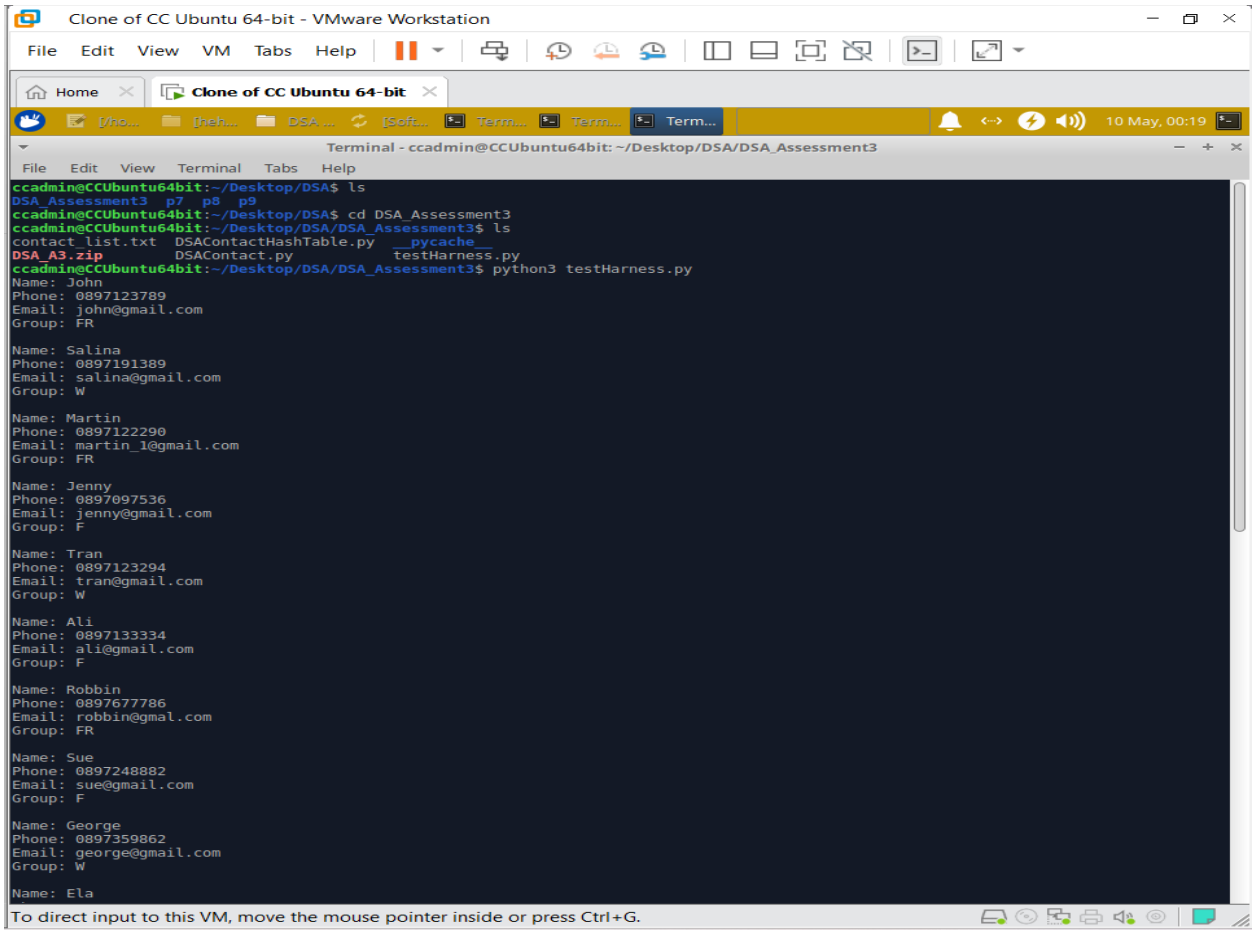
Comments for Complex Logic

The presence of comments throughout the code provides clear explanations of complex logic, algorithms, and data structures. For example, the comments explain the process

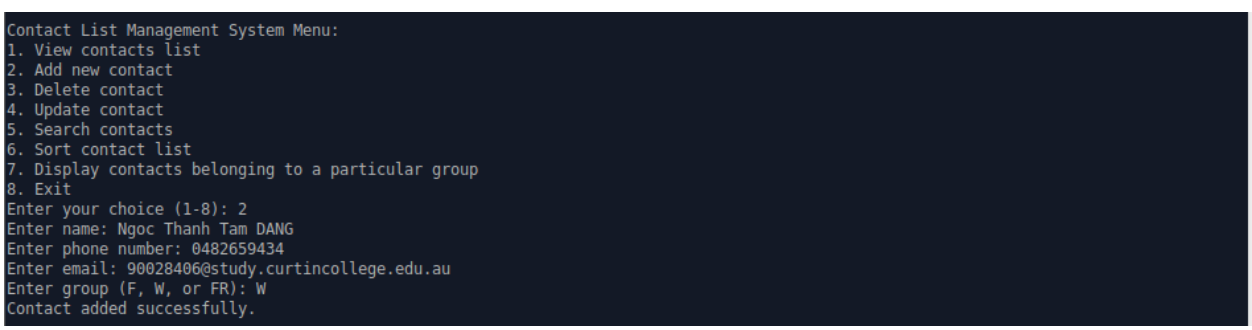
of adding, deleting, updating, and searching for contacts in a hash table. This approach improves code understanding for both developers and maintainers.

10. Result

When execute the code



When add new contact



When search contact

```
Contact List Management System Menu:
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 5
Enter search term (name or phone number): Tam
Matching contacts found:
Name: Ngoc Thanh Tam DANG
Phone: 0482659434
Email: 90028406@study.curtincollege.edu.au
Group: W
```

When sort contact by name from A-Z

```
Contact List Management System Menu:
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 6

Sorted Contacts List:
Name: Ali
Phone: 0897133334
Email: ali@gmail.com
Group: F

Name: Ela
Phone: 0897136742
Email: ela@gmail.com
Group: W

Name: Faraz
Phone: 0897189273
Email: faraz@gmail.com
Group: W

Name: George
Phone: 0897359862
Email: george@gmail.com
Group: W

Name: George
Phone: 0897443200
Email: george_1@gmail.com
Group: F

Name: George
Phone: 0897359862
Email: george@gmail.com
Group: W

Name: George
Phone: 0897443200
Email: george_1@gmail.com
Group: F

Name: Jenny
Phone: 0897097536
Email: jenny@gmail.com
Group: F

Name: John
```

When display contact by organizing the group

```
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 7
Enter group to display (F, W, or FR): F

Contacts belonging to group 'F':
Name: Jenny
Phone: 0897097536
Email: jenny@gmail.com
Group: F

Name: Sue
Phone: 0897248882
Email: sue@gmail.com
Group: F

Name: George
Phone: 0897443200
Email: george_1@gmail.com
Group: F

Name: Ali
Phone: 0897133334
Email: ali@gmail.com
Group: F

Name: Smith
Phone: 0897121283
Email: smith@gmail.com
Group: F

Name: Martin
Phone: 0897022768
Email: martin@gmail.com
Group: F
```

When update the contact

```
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 4
Enter phone number of contact to update: 0482659434
Enter updated name: Tammy
Enter updated email: dnttamm@gmail.com
Enter updated group (F, W, or FR): FR
Contact updated successfully.
```

When delete contact

```
Contact List Management System Menu:
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 3
Enter phone number of contact to delete: 0482659434
Contact deleted successfully.
```

When search the deleted contact

```
Contact List Management System Menu:
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 5
Enter search term (name or phone number): Tammy
No matching contacts found.
```

When the end execute code

```
Contact List Management System Menu:
1. View contacts list
2. Add new contact
3. Delete contact
4. Update contact
5. Search contacts
6. Sort contact list
7. Display contacts belonging to a particular group
8. Exit
Enter your choice (1-8): 8
Exiting program...
ccadmin@CCUbuntu64bit:~/Desktop/DSA/DSA_Assessment3$
```